

LAB 05_REPORT

Programming paradigms



NOVEMBER 9, 2026

NAME: SIEM KHATTYAVATEY
Student ID: p20250056
GIT URL: https://github.com/zikkyiscool/PDI.git

Practice 1:

Problem: Write a procedural program to find the sum of even numbers from 1 to 100. Display result on screen.

Solution:

```java
class Exercise_1{
    public static void main(String[] args){
        int sum = 0;
        int num;
        for(int i = 1 ; i<=100 ;i++){
            if(i%2 == 0){
                num = i ;
                sum += num;
            }
        }
        System.out.println("The sum of even number from 1-100 is " + sum );
    }
}
```

Output:

```
The sum of even number from 1-100 is 2550
PS D:\Year 2\PDI\Lab05>
```

Practice 2:

Problem: Implement a calculator that supports basic arithmetic operations (+, -, *, /, sum from 1 to n) using a menu-driven approach.

Solution:

```java
import java.util.Scanner;

public class Exercise_2 {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int num_1;
        int num_2;
        float result = 0;
```

```java

        System.out.println("Please input number from 1 to 6");
        System.out.println("1.Addition (+)");
        System.out.println("2.Subtraction (-)");
        System.out.println("3.Multiplication (x)");
        System.out.println("4.Division (/)");
        System.out.println("5.Find sum of x number");
        System.out.println("6.Exit");
        System.out.print("Your choice: ");
        int choice = sc.nextInt();

        switch (choice) {
            case 1:
                System.out.print("Please input first number: ");
                num_1 = sc.nextInt();
                System.out.print("Please input second number: ");
                num_2 = sc.nextInt();
                result = num_1 + num_2 ;
                System.out.println("The result is " + result);
                System.out.println("Thank you for using our program!");
                break;

            case 2:
                System.out.print("Please input first number: ");
                num_1 = sc.nextInt();
                System.out.print("Please input second number: ");
                num_2 = sc.nextInt();
                result = num_1 - num_2 ;
                System.out.println("The result is " + result);
                System.out.println("Thank you for using our program!");
                break;

            case 3:
                System.out.print("Please input first number: ");
                num_1 = sc.nextInt();
                System.out.print("Please input second number: ");
                num_2 = sc.nextInt();
                result = num_1 * num_2 ;
                System.out.println("The result is " + result);
                System.out.println("Thank you for using our program!");
                break;

```

```java
                case 4:
                    System.out.print("Please input first number: ");
                    num_1 = sc.nextInt();
                    System.out.print("Please input second number: ");
                    num_2 = sc.nextInt();
                    if (num_2 != 0) {
                            result = (float) num_1 / num_2;
                            System.out.println("The result is " + result);
                            System.out.println("Thank you for using our program!");
                        } else {
                            System.out.println("You cannot divide by zero you big donut!");
                        }
                    break;

                case 5:
                    System.out.print("Please input number: ");
                    num_1 = sc.nextInt();
                    for(int i = 1 ; i<=num_1 ;i++){
                        result += i;
                    }
                    System.out.println("The result is " + result);
                    break;

                case 6:
                    System.out.println("Thank you for using our program!");
                    break;

                default:
                    System.out.println("Please input from 1 to 6");
                    break;
            }

        }
    }
```

Output:

```
Please input number from 1 to 6
1.Addition (+)
2.Subtraction (-)
3.Multiplication (x)
4.Division (/)
5.Find sum of x number
6.Exit
Your choice: 4
Please input first number: 2
Please input second number: 3
The result is 0.6666667
Thank you for using our program!
PS D:\Year 2\PDI\Lab05>
```

```
Please input number from 1 to 6
1.Addition (+)
2.Subtraction (-)
3.Multiplication (x)
4.Division (/)
5.Find sum of x number
6.Exit
Your choice: 5
Please input number: 6
The result is 21.0
PS D:\Year 2\PDI\Lab05>
```

```
Please input number from 1 to 6
1.Addition (+)
2.Subtraction (-)
3.Multiplication (x)
4.Division (/)
5.Find sum of x number
6.Exit
Your choice: 4
Please input first number: 2
Please input second number: 0
You cannot divide by zero you big donut!
PS D:\Year 2\PDI\Lab05>
```

Practice 3:

Problem: Creates two Book objects using default and parameterized constructors, then displays their title, author, and price.

Solution:

Book.java: Defines attributes, constructors, and display method

```java
package Exercise_3;
public class Book {
    //Attribute
    String title;
    String author;
    double price;

    //Constructor for default
    public Book(){
        title = "Unknown";
        author = "Unknown";
        price = 0;
    }

    //Parameter connstructor
    public Book(String title, String author, double price){
        this.title = title;
        this.author = author;
        this.price = price;
    }

    //Method to display detail of book
    public void display(){
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Price: " + price);
    }
}
```

Main.java: Main program creates objects.

```java
package Exercise_3;
public class Main {
    Run | Debug
    public static void main(String[]args){
        Book book1 = new Book();
        Book book2 = new Book(title: "King's game", author: "J.Art", price: 27);

        //Display books
        System.out.println("=======Book Information=======");
        book1.display();
        System.out.println("------------------------------");
        book2.display();
        System.out.println("------------------------------");
    }
}
```

Output:

```
=======Book Information=======
Title: Unknown
Author: Unknown
Price: 0.0
------------------------------
Title: King's game
Author: J.Art
Price: 27.0
------------------------------
PS D:\Year 2\PDI\Lab05>
```

Practice 4:

Problem: Creates an Employee object using a constructor, with getters and setters to access and update its private attributes.

Solution:

Employee.java: Defines the employee's information and manages its attributes.

Employee.java > ...

```java
public class Employee {

    //Attributes
    private String id;
    private String name;
    private String position;
    private double salary;
    private String department;

    // Constructor
    public Employee(String id, String name, String position,
                    double salary, String department) {
        this.id = id;
        this.name = name;
        this.position = position;
        this.salary = salary;
        this.department = department;
    }

    // Getter methods
    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getPosition() {
        return position;
    }

    public double getSalary() {
        return salary;
    }

    public String getDepartment() {
        return department;
    }

```

```java
    //Setter methods
    public void setId(String id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setPosition(String position) {
        this.position = position;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    //Display employee information
    public void display() {
        System.out.println("Employee ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Position: " + position);
        System.out.println("Salary: $" + salary);
        System.out.println("Department: " + department);
    }
}
```

Main.java: Creates and tests the Employee object.

```java
public class Main {
    Run | Debug
    public static void main(String[] args) {
        Employee emp1 = new Employee(
            id: "101", name: "John", position: "Manager", salary: 1500, department: "IT");
        Employee emp2 = new Employee(
            id: "507", name: "Vorn Bopha", position: "Saleswomen", salary: 600, department: "Sale;");

        // Display employee information
        System.out.println("===== Employee Information =====");
        emp1.display();
        emp1.setSalary(salary: 1800);
        System.out.println("\nUpdated Salary: $" + emp1.getSalary());
        System.out.println("--------------------------------");

        emp2.display();
        emp2.setSalary(salary: 650);
        System.out.println("\nUpdated Salary: $" + emp2.getSalary());
        System.out.println("--------------------------------");
    }
}
```

Output:

```
===== Employee Information =====
Employee ID: 101
Name: John
Position: Manager
Salary: $1500.0
Department: IT

Updated Salary: $1800.0
--------------------------------
```

```
Employee ID: 507
Name: Vorn Bopha
Position: Saleswomen
Salary: $600.0
Department: Sale;

Updated Salary: $650.0
--------------------------------
PS D:\Year 2\PDI\Lab05>
```

Practice 5:

Problem: Compare the execution of a factorial function implemented procedurally and using recursion.

Solution:

```java
import java.math.BigInteger;

public class Exercise_5 {

    // Procedural method using for loop
    public static BigInteger factorialLoop(int n) {
        BigInteger result = BigInteger.ONE;

        for (int i = 1; i <= n; i++) {
            result = result.multiply(BigInteger.valueOf(i));
        }

        return result;
    }

    // Recursive method
    public static BigInteger factorialRecursive(int n) {
        if (n == 0 || n == 1) {
            return BigInteger.ONE;
        }

        return BigInteger.valueOf(n).multiply(factorialRecursive(n - 1));
    }

    Run | Debug
    public static void main(String[] args) {

        int[] numbers = {3, 6, 10, 50};

        for (int n : numbers) {
            System.out.println("Factorial of " + n);

            System.out.println("Procedural: " + factorialLoop(n));

            System.out.println("Recursive: " + factorialRecursive(n));

            System.out.println("--------------------------");
        }
    }
}
```

Output:

```
Factorial of 3
Procedural: 6
Recursive: 6
--------------------------
Factorial of 6
Procedural: 720
Recursive: 720
--------------------------
Factorial of 10
Procedural: 3628800
Recursive: 3628800
--------------------------
Factorial of 50
Procedural: 30414093201713378043612608166064768844377641568960512000000000000
Recursive: 30414093201713378043612608166064768844377641568960512000000000000
--------------------------
```

Practice 6:

Problem: Create Rectangle and Circle classes using encapsulation, constructors, getters, and setters to manage dimensions and calculate area and perimeter.

Solution:

Rectangle.java: Create methods to calculate area and perimeter

Rectangle.java > ...

```java
public class Rectangle {

    // Attribute
        private double width;
        private double height;

    // Constructor
    public Rectangle(double width, double height){
        this.width = width;
        this.height = height;
    }

    // Getter methods
    public double getWidth(){
        return width;
    }
    public double getHeight(){
        return height;
    }

    // Setter methods
    public void setWidth(double width){
        this.width = width;
    }
    public void setLength(double height){
        this.height = height;
    }

    // Method to calculate area of rectangle
    public double rectangleArea(){
        return (width * height);
    }

    // Method to calculate parimeter of rectangle
    public double rectangleParimeter(){
        return (2*(width + height));
    }
}
```

Circle.java: Create methods to calculate area and perimeter of circle.

```java
public class Circle {

    // Getter method
    public double getRadius(){
        return radius;
    }

    // Setter method
    public void setRadius(double radius){
        this.radius = radius;
    }

    // Method to calculate parimeter
    public double circleParimeter(){
        return (2 * 3.14 * radius);
    }

    // Method to calculate parimeter
    public double circleArea(){
        return (2 * 3.14 * radius);
    }

}
```

Main.java: Create objects and tests the methods from both classes.

```java
public class Main {
    Run | Debug
    public static void main(String[] args) {

        Rectangle rectangle = new Rectangle(width: 5, height: 10);

        System.out.println("===== Rectangle =====");
        System.out.println("Width: " + rectangle.getWidth());
        System.out.println("Height: " + rectangle.getHeight());
        System.out.println("Area: " + rectangle.rectangleArea());
        System.out.println("Perimeter: " + rectangle.rectangleParimeter());
        Circle circle = new Circle(radius: 7);

        System.out.println("\n===== Circle =====");
        System.out.println("Radius: " + circle.getRadius());
        System.out.println("Area: " + circle.circleArea());
        System.out.println("Perimeter: " + circle.circleParimeter());

        rectangle.setWidth(width: 8);
        circle.setRadius(radius: 10);

        System.out.println("\n===== Updated Values =====");
        System.out.println("Rectangle Area: " + rectangle.rectangleArea());
        System.out.println("Circle Area: " + circle.circleArea());
    }
}
```

Output:

```
===== Rectangle =====
Width: 5.0
Height: 10.0
Area: 50.0
Perimeter: 30.0

===== Circle =====
Radius: 7.0
Area: 43.96
Perimeter: 43.96

===== Updated Values =====
Rectangle Area: 80.0
Circle Area: 62.800000000000004
PS D:\Year 2\PDI\Lab05>
```

Practice 7:

Problem: Write a program that can creates and runs multiple threads using Thread class. Let's define 3 functions to: find sum from 1 to n, show all prime numbers in between 1 to 100, and read data from a CSV file of student list (name, ID, age, gender) and store it into an array list.

Solution:

Student.java: Defines the structure of a student object.

Exercise_7 > J Student.java > ...

```java
package Exercise_7;
public class Student {
    private String name;
    private int id;
    private int age;
    private String gender;

    public Student(String name, int id, int age, String gender) {
        this.name = name;
        this.id = id;
        this.age = age;
        this.gender = gender;
    }

    public void display() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("Age: " + age);
        System.out.println("Gender: " + gender);
        System.out.println("--------------------");
    }
}
```

PrimeThread.java: Finds all prime numbers between 1 and 100.

Exercise_7 > PrimeThread.java > ...

```java
package Exercise_7;

public class PrimeThread extends Thread {

    @Override
    public void run() {
        System.out.println("Prime numbers from 1 to 100:");

        for (int i = 2; i <= 100; i++) {
            int count = 0;

            for (int j = 1; j <= i; j++) {
                if (i % j == 0) {
                    count++;
                }
            }

            if (count == 2) {
                System.out.print(i + " ");
            }
        }

        System.out.println();
    }
}
```

SumThread.java: Calculates the sum of numbers from 1 to a given number.

Exercise_7 > SumThread.java > ...

```java
package Exercise_7;
public class SumThread extends Thread {

    private int n;

    public SumThread(int n) {
        this.n = n;
    }

    @Override
    public void run() {
        int sum = 0;

        for (int i = 1; i <= n; i++) {
            sum += i;
        }

        System.out.println("Sum from 1 to " + n + " = " + sum);
    }
}
```

StudentCSVThread.java: Reads student information from students.csv and displays it.

Exercise_7 > StudentCSVThread.java > StudentCSVThread > run()

```java
package Exercise_7;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;

public class StudentCSVThread extends Thread {

    @Override
    public void run() {

        ArrayList<Student> students = new ArrayList<>();

        try {
            BufferedReader br = new BufferedReader(
                new FileReader("students.csv")
            );
            String line;

            // Skip header
            br.readLine();
            while ((line = br.readLine()) != null) {

            // Skip empty lines
            if (line.trim().isEmpty()) {
                continue;
            }

            String[] data = line.split(",");

            if (data.length != 4) {
                System.out.println("Invalid CSV row: " + line);
                continue;
            }

            String name = data[0];
            int id = Integer.parseInt(data[1]);
            int age = Integer.parseInt(data[2]);
            String gender = data[3];

            Student student = new Student(name, id, age, gender);

            students.add(student);
            }
            br.close();

            System.out.println("\nStudent Information:");

            for (Student s : students) {
                s.display();
            }
            System.out.println("Total students: " + students.size());

        } catch (IOException e) {
            System.out.println("Error reading CSV file: " + e.getMessage());
        }
    }
}
```

Student.csv: StudentCSVThread.java reads the file and processes its information.

students.csv > data

```
name,id,age,gender
Puthi,101,20,Male
Bopha,102,21,Female
Davin,103,19,Male
```

Main.java: Creates and starts the three threads.

Exercise_7 > Main.java > Main

```java
package Exercise_7;

public class Main {
    Run | Debug
    public static void main(String[] args) {

        SumThread t1 = new SumThread(n: 10);
        PrimeThread t2 = new PrimeThread();
        StudentCSVThread t3 = new StudentCSVThread();

        t1.start();

        try {
            t1.join();
        } catch (InterruptedException e) {
            System.out.println(e.getMessage());
        }

        t2.start();

        try {
            t2.join();
        } catch (InterruptedException e) {
            System.out.println(e.getMessage());
        }

        t3.start();
    }
}
```

Output:

```
Sum from 1 to 10 = 55
Prime numbers from 1 to 100:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

Student Information:
Name: Puthi
ID: 101
Age: 20
Gender: Male
--------------------
Name: Bopha
ID: 102
Age: 21
Gender: Female
--------------------
Name: Davin
ID: 103
Age: 19
Gender: Male
--------------------
Total students: 3
PS D:\Year 2\PDI\Lab05>
```

Practice 8:

Problem: Write a program that can creates and runs multiple threads using Thread class. Let's define 3 functions to: find sum from 1 to n, show all prime numbers in between 1 to 100, and read data from a CSV file of student list (name, ID, age, gender) and store it into an array list.

Solution:

Student.java: Represents an individual student and stores their information.

Exercise_8 > Student.java > ...

```java
package Exercise_8;

public class Student {
    private String name;
    private int id;
    private int age;
    private String gender;

    public Student(String name, int id, int age, String gender) {
        this.name = name;
        this.id = id;
        this.age = age;
        this.gender = gender;
    }

    public void display() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("Age: " + age);
        System.out.println("Gender: " + gender);
        System.out.println("--------------------");
    }
}
```

PrimeTask.java: Finds and displays prime numbers from 1 to 100.

Exercise_8 > PrimeTask.java > ...

```java
package Exercise_8;

public class PrimeTask implements Runnable {

    @Override
    public void run() {
        System.out.println("Prime numbers from 1 to 100:");

        for (int i = 2; i <= 100; i++) {
            int count = 0;

            for (int j = 1; j <= i; j++) {
                if (i % j == 0) {
                    count++;
                }
            }

            if (count == 2) {
                System.out.print(i + " ");
            }
        }

        System.out.println();
    }
}
```

SumTask.java: Calculates the sum of numbers from 1 to a given number.

Exercise_8 > SumTask.java > ...

```java
package Exercise_8;

public class SumTask implements Runnable {

    private int n;

    public SumTask(int n) {
        this.n = n;
    }

    @Override
    public void run() {
        int sum = 0;

        for (int i = 1; i <= n; i++) {
            sum += i;
        }

        System.out.println("Sum from 1 to " + n + " = " + sum);
    }
}
```

Main.java: Creates and controls the execution of three tasks using the Runnable interface.

Exercise_8 > Main.java > ...

```java
package Exercise_8;

public class Main {
    Run | Debug
    public static void main(String[] args) {

        SumTask task1 = new SumTask(n: 10);
        PrimeTask task2 = new PrimeTask();
        StudentCSVTask task3 = new StudentCSVTask();

        Thread t1 = new Thread(task1);
        Thread t2 = new Thread(task2);
        Thread t3 = new Thread(task3);

        t1.start();

        try {
            t1.join();
        } catch (InterruptedException e) {
            System.out.println(e.getMessage());
        }

        t2.start();

        try {
            t2.join();
        } catch (InterruptedException e) {
            System.out.println(e.getMessage());
        }

        t3.start();
    }
}
```

Student.csv: StudentCSVThread.java reads the file and processes its information.

students.csv > data

```
name,id,age,gender
Puthi,101,20,Male
Bopha,102,21,Female
Davin,103,19,Male
```

StudentCSVTask.java: Reads student information from students.csv and displays it.

Exercise_8 > StudentCSVTask.java > ...

```java
package Exercise_8;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;

public class StudentCSVTask extends Thread {

    @Override
    public void run() {

        ArrayList<Student> students = new ArrayList<>();

        try {
            BufferedReader br = new BufferedReader(
                new FileReader("students.csv")
            );
            String line;

            // Skip header
            br.readLine();
            while ((line = br.readLine()) != null) {

            // Skip empty lines
            if (line.trim().isEmpty()) {
                continue;
            }

            String[] data = line.split(",");

            if (data.length != 4) {
                System.out.println("Invalid CSV row: " + line);
                continue;
            }

            String name = data[0];
            int id = Integer.parseInt(data[1]);
            int age = Integer.parseInt(data[2]);
            String gender = data[3];

            Student student = new Student(name, id, age, gender);

            students.add(student);
            }
            br.close();

            System.out.println("\nStudent Information:");

            for (Student s : students) {
                s.display();
            }
            System.out.println("Total students: " + students.size());

        } catch (IOException e) {
            System.out.println("Error reading CSV file: " + e.getMessage());
        }
    }
}
```

Output:

```
Sum from 1 to 10 = 55
Prime numbers from 1 to 100:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

Student Information:
Name: Puthi
ID: 101
Age: 20
Gender: Male
--------------------
Name: Bopha
ID: 102
Age: 21
Gender: Female
--------------------
Name: Davin
ID: 103
Age: 19
Gender: Male
--------------------
Total students: 3
PS D:\Year 2\PDI\Lab05>
```

Practice 9:

Problem: Write a functional program to:

a) find the cube of each number in a list and print the results

b) filter out positive numbers from a given list and print them.

c) sort data in a list.

Solution:

```java
import java.util.Arrays;
import java.util.List;

public class Exercise_9 {
    Run | Debug
    public static void main(String[]args){
        List<Integer> numbers = Arrays.asList(5, 2, 9, -1, 3, -8, 20, 50);

        // a.find cube of each number
        System.out.print("The cube of each number is: ");
        numbers.stream()
                .map(n->n*n*n)
                .forEach(n -> System.out.print(n + " "));

        // b.filter out positive number
        System.out.print("\n\nPositive number: ");
        numbers.stream()
                .filter(n -> n > 0)
                .forEach(n ->System.out.print(n + " "));

        // c.sort data in a list
        System.out.print("\n\nSorted data: ");
        numbers.stream()
                .sorted()
                .forEach(n -> System.out.print(n + " "));
    }
}
```

Output:

```
The cube of each number is: 125 8 729 -1 27 -512 8000 125000

Positive number: 5 2 9 3 20 50

Sorted data: -8 -1 2 3 5 9 20 50
PS D:\Year 2\PDI\Lab05>
```

Practice 10:

Problem: Read students' names from a CSV file, then display them in A-Z order.

Solution:

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;

public class Exercise_10 {
    Run | Debug
    public static void main(String[]args){

        ArrayList<String> students = new ArrayList<>();
        try(BufferedReader br = new BufferedReader(new FileReader("students.csv"))){
            String line;
            br.readLine();
        while ((line = br.readLine()) != null) {

                String[] data = line.split(",");

                students.add(data[0]);
            }
        System.out.print("Student sorted in order");
        students.stream()
                .sorted(String.CASE_INSENSITIVE_ORDER)
                .forEach(name -> System.out.println(name));
       } catch (IOException e) {
            System.out.println("Error reading CSV file: " + e.getMessage());
        }
    }
}
```

Output:

```
Student sorted in order

Bopha
Davin
Puthi
PS D:\Year 2\PDI\Lab05>
```

Practice 11:

Problem: Find max and min in an array using functional programming.

Solution:

```java
import java.util.stream.IntStream;

public class Exercise_11 {
    Run | Debug
    public static void main(String[]args){
        int[ ] numbers = {5, 2, 9, 1, 3, 8};

        int max = IntStream.of(numbers)
                           .max()
                           .getAsInt();

        int min = IntStream.of(numbers)
                           .min()
                           .getAsInt();

        System.out.println("Max number: " + max);
        System.out.println("Min number: " + min);
    }
}
```

Output:

```
Max number: 9
Min number: 1
PS D:\Year 2\PDI\Lab05>
```

THE END